

Modul Praktikum 4

INHERITANCE, GENERALIZATION & OVERRIDING



D3 Teknik Informatika

POLITEKNIK NEGERI BANDUNG

Daftar Isi

Daftar Isi..... 2

1. Modul Overview..... 3

2. Persiapan Praktikum 3

3. Praktikum I - Circle dan Cylinder 4

4. Praktikum II - Shape, Circle, Rectangle, dan Square 9

5. Kembangkan Kasus Tugas Sebelumnya 13

6. Laporan dan Pengumpulan 14

7. Checkpoint dan Refleksi 14

1. Modul Overview

Pada pertemuan ini Anda tidak memulai dari definisi panjang. Anda akan membangun beberapa class, menjalankan program, mengubah satu bagian, mengamati efeknya, lalu menuliskan penjelasan berdasarkan bukti dari code dan output. Istilah teori akan dirapikan setelah praktikum.

1.1 Tujuan Pembelajaran

1. Menjelaskan hubungan superclass-subclass berdasarkan code yang benar-benar dijalankan.
2. Menerapkan inheritance dengan extends tanpa menggandakan state dan behavior yang sudah dimiliki superclass.
3. Menggunakan constructor subclass dan super(...) untuk menginisialisasi bagian object yang berasal dari superclass.
4. Membedakan inherited method dan overridden method melalui perubahan output program.
5. Menggunakan @Override sebagai pemeriksaan compiler terhadap niat overriding.
6. Menggunakan super.method(...) ketika subclass tetap membutuhkan behavior milik superclass.
7. Menjaga encapsulation pada hierarchy class: field private tetap diakses melalui behavior yang disediakan class pemiliknya.
8. Menerapkan inheritance, override, dan super pada kasus tugas sebelumnya.

1.2 Cara Kerja di Setiap Langkah

PREDICT -> CODE -> RUN -> OBSERVE -> EXPLAIN

Sebelum menjalankan perubahan penting, tulis dugaan singkat. Setelah run, simpan output/error. Jangan langsung memperbaiki error tanpa mencatat apa yang terjadi.

1.3 Batas Praktikum

1. Fokus wajib: inheritance, extends, constructor chain, super, inherited method, overriding, @Override, dan generalization-specialization melalui kasus.
2. Abstract class, interface, dan multiple inheritance bukan target wajib pada modul ini.
3. Tidak menggunakan database, GUI, framework, atau library eksternal.
4. Jangan mengubah field private menjadi public hanya agar subclass lebih mudah mengaksesnya.

1.4 Penggunaan AI

Diagnosis pertama harus dilakukan sendiri. AI boleh digunakan setelah Anda memiliki bukti compile/run untuk meminta hint atau penjelasan error. Jangan meminta AI menghasilkan seluruh task. Catat bantuan yang digunakan dan verifikasi kembali dengan menjalankan code.

2. Persiapan Praktikum

2.1 Struktur Project

```
praktikum4/
├── src/
│   ├── Circle.java
│   ├── Cylinder.java
│   ├── TestCylinder.java
│   ├── Shape.java
│   └── Rectangle.java
```

```
|   | Square.java
|   | TestShape.java
|___| laporan/
```

Gunakan package yang konsisten jika project Anda memakai package. Jangan mencampur file dengan package berbeda dalam satu folder source.

2.2 Checklist Awal

- JDK dan IDE dapat compile/run project Java sederhana.
- Buat project baru dan pastikan Hello World berjalan.
- Buat folder laporan untuk menyimpan screenshot/output penting.
- Siapkan catatan singkat untuk menulis prediksi dan observasi.

Jangan mengejar output akhir secepat mungkin.

Nilai pembelajaran ada pada perubahan kecil yang Anda lakukan dan kemampuan menjelaskan mengapa output/error berubah.

3. Praktikum I - Circle dan Cylinder

Tujuan blok pertama adalah membangun hubungan Circle -> Cylinder secara bertahap. Jangan lompat ke code akhir. Jalankan setiap checkpoint karena beberapa konsep hanya terlihat setelah code diubah.

Step 1 - Bangun Circle versi paling sederhana (10 menit)

Buat Circle.java dengan state radius dan color, dua constructor, getRadius(), getArea(), dan toString().

```
public class Circle {
    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }

    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }

    public double getRadius() {
        return radius;
    }

    public double getArea() {
        return radius * radius * Math.PI;
    }

    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", color=" + color + "];"
    }
}
```

Buat TestCircle sementara, instantiate Circle() dan Circle(2.5), lalu cetak getRadius(), getArea(), dan object-nya.

Checkpoint 1

Tuliskan: mana state Circle? mana behavior Circle? apa perbedaan state dari dua object yang dibuat?

Step 2 - Lengkapi encapsulation Circle (10 menit)

Tambahkan constructor yang menerima radius dan color, getter/setter color, serta setter radius.

```
public Circle(double radius, String color) {
    this.radius = radius;
    this.color = color;
}

public void setRadius(double radius) {
    this.radius = radius;
}

public String getColor() {
    return color;
}

public void setColor(String color) {
    this.color = color;
}
```

- Buat Circle baru dengan radius 3.0 dan color "blue".
- Ubah radius dan color melalui setter, kemudian cetak ulang object.
- Jangan mengubah field private menjadi public.

Checkpoint 2

Mengapa code di luar Circle tidak mengubah radius secara langsung? Apa peran method public terhadap state private?

Step 3 - Prediksi sebelum membuat Cylinder (5 menit)

Cylinder mempunyai alas berbentuk lingkaran dan menambahkan height. Sebelum coding, jawab singkat:

- Apakah radius dan color perlu ditulis ulang sebagai field Cylinder?
- Behavior apa dari Circle yang masih berguna untuk Cylinder?
- State apa yang hanya dimiliki Cylinder?

Step 4 - Buat Cylinder sebagai subclass Circle (15 menit)

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }
}
```

```

    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
}

```

Jangan tambahkan radius dan color ke Cylinder.

Tujuan langkah ini adalah menguji apa yang tersedia karena Cylinder extends Circle.

Step 5 - Uji inherited method (10 menit)

```

public class TestCylinder {
    public static void main(String[] args) {
        Cylinder c1 = new Cylinder();
        Cylinder c2 = new Cylinder(10.0);
        Cylinder c3 = new Cylinder(2.0, 10.0);

        printCylinder("c1", c1);
        printCylinder("c2", c2);
        printCylinder("c3", c3);
    }

    private static void printCylinder(String name, Cylinder c) {
        System.out.println(name);
        System.out.println("radius      = " + c.getRadius());
        System.out.println("height     = " + c.getHeight());
        System.out.println("base area  = " + c.getArea());
        System.out.println("volume     = " + c.getVolume());
        System.out.println();
    }
}

```

- Run program dan simpan output.
- Di Cylinder.java, beri tanda pada method yang benar-benar ditulis di Cylinder.

- Bandingkan dengan method yang dapat dipanggil pada object Cylinder tetapi tidak ditulis di Cylinder.

Checkpoint 3

Jelaskan berdasarkan code: mengapa `c3.getRadius()` dan `c3.getArea()` dapat dipanggil padahal method tersebut tidak ditulis ulang pada Cylinder?

Step 6 - Amati constructor chain dan fungsi `super(...)` (15 menit)

Tambahkan output sementara di setiap constructor.

```
// contoh di Circle
public Circle(double radius) {
    System.out.println("Circle(double) dipanggil");
    this.radius = radius;
    color = "red";
}

// contoh di Cylinder
public Cylinder(double radius, double height) {
    super(radius);
    System.out.println("Cylinder(double,double) dipanggil");
    this.height = height;
}
```

- Jalankan hanya `new Cylinder(2.0, 10.0)`. Catat urutan output constructor.
- Hapus sementara `super(radius)` dari constructor `Cylinder(double,double)`, lalu compile.
- Catat error/perubahan yang terjadi. Setelah itu kembalikan code ke kondisi benar.
- Hapus kembali `println` sementara setelah observasi selesai.

Checkpoint 4

Apa yang dikerjakan lebih dahulu saat object Cylinder dibuat: bagian Circle atau bagian Cylinder? Bukti apa dari output Anda?

Step 7 - Uji batas encapsulation pada subclass (5 menit)

Di dalam Cylinder, coba tulis method sementara berikut lalu compile.

```
public double cobaAksesRadius() {
    return radius; // radius milik Circle dan private
}
```

- Simpan pesan error compiler.
- Hapus method tersebut atau perbaiki dengan `getRadius()`.

Checkpoint 5

Lengkapi kalimat: extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi ...

Step 8 - Override getArea(), lalu temukan bug pada getVolume() (15 menit)

Sekarang ubah makna getArea() pada Cylinder menjadi luas permukaan silinder.

```
@Override
public double getArea() {
    return 2 * Math.PI * getRadius() * height
        + 2 * Math.PI * getRadius() * getRadius();
}
```

- Run TestCylinder. Bandingkan getArea() sebelum dan sesudah override.
- Periksa getVolume(). Apakah hasilnya masih menggunakan luas alas?
- Jangan langsung memperbaiki. Jelaskan dulu method getArea() versi mana yang dipanggil dari getVolume().

Perbaiki getVolume() dengan meminta secara eksplisit implementasi Circle:

```
public double getVolume() {
    return super.getArea() * height;
}
```

- Run kembali dan bandingkan hasilnya.

Checkpoint 6 - Aha moment

getArea() pada object Cylinder sekarang berbeda dari getArea() pada Circle. super.getArea() dipakai ketika code di subclass secara sengaja ingin menggunakan implementasi milik superclass.

Step 9 - Override toString() dan uji @Override (10 menit)

```
@Override
public String toString() {
    return "Cylinder: subclass of " + super.toString()
        + ", height=" + height;
}
```

- Cetak System.out.println(c3).
- Ubah sementara nama method menjadi toStrng(), tetapi pertahankan @Override. Compile dan simpan error.
- Hapus @Override lalu compile lagi. Perhatikan perbedaannya.
- Kembalikan method ke toString() dan pasang @Override.

Checkpoint 7

Tuliskan dengan bahasa Anda: @Override tidak membuat inheritance terjadi. Lalu apa kegunaannya?

4. Praktikum II - Shape, Circle, Rectangle, dan Square

Blok kedua memperluas pengalaman Anda. Kali ini beberapa class memiliki state dan behavior yang sama. Anda akan menempatkan kemiripan pada Shape, lalu mempertahankan perbedaan pada subclass.

Step 10 - Buat superclass Shape (10 menit)

```
public class Shape {
    private String color;
    private boolean filled;

    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    @Override
    public String toString() {
        return "Shape[color=" + color + ", filled=" + filled + "]";
    }
}
```

- Buat satu object Shape default dan satu object Shape dengan parameter.
- Cetak keduanya untuk memastikan constructor bekerja.

Checkpoint 8

State apa yang cukup umum sehingga masuk akal dimiliki Circle maupun Rectangle?

Step 11 - Buat Circle sebagai specialization dari Shape (10 menit)

```
public class Circle extends Shape {
    private double radius;

    public Circle() {
        super();
        radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }
}
```

```

public Circle(double radius, String color, boolean filled) {
    super(color, filled);
    this.radius = radius;
}

public double getRadius() { return radius; }
public void setRadius(double radius) { this.radius = radius; }

public double getArea() { return Math.PI * radius * radius; }
public double getPerimeter() { return 2 * Math.PI * radius; }

@Override
public String toString() {
    return "Circle[radius=" + radius + ", " + super.toString() + "];"
}
}

```

- Instantiate Circle dengan constructor berbeda.
- Cetak area, perimeter, color, filled, dan toString().

Checkpoint 9

Pisahkan daftar berikut menjadi dua: behavior yang berasal dari Shape dan behavior yang spesifik milik Circle.

Step 12 - Buat Rectangle sebagai specialization dari Shape (10 menit)

```

public class Rectangle extends Shape {
    private double width;
    private double length;

    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }

    public double getArea() { return width * length; }
}

```

```

    public double getPerimeter() { return 2 * (width + length); }

    @Override
    public String toString() {
        return "Rectangle[width=" + width + ", length=" + length
            + ", " + super.toString() + "]\n";
    }
}

```

- Instantiate Rectangle dan uji constructor parameter.
- Cetak area, perimeter, dan toString().

Checkpoint 10

Apa kemiripan Circle dan Rectangle yang sudah ditarik ke Shape? Apa yang sengaja tetap berbeda?

Step 13 - Buat Square dari Rectangle (10 menit)

Square tidak menambah field baru. Ia menggunakan width dan length dari Rectangle, tetapi harus menjaga keduanya selalu sama.

```

public class Square extends Rectangle {
    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square[side=" + getSide() + ", " + super.toString() + "]\n";
    }
}

```

Belum override setWidth() dan setLength(). Kita akan sengaja mengujinya terlebih dahulu.

Step 14 - Temukan pelanggaran aturan Square (10 menit)

```

Square s = new Square(5.0);
System.out.println("awal : " + s);

```

```
System.out.println("area    : " + s.getArea());

s.setWidth(8.0);
System.out.println("sesudah setWidth(8): " + s);
```

- Jalankan code di atas.
- Periksa width dan length setelah setWidth(8).
- Jawab: apakah object tersebut masih merepresentasikan Square?

Checkpoint 11

Inheritance dapat memberikan behavior yang secara teknis tersedia tetapi tidak selalu sesuai dengan aturan subclass. Di sinilah overriding dapat menjaga aturan object yang lebih spesifik.

Step 15 - Override setter agar Square tetap valid (10 menit)

```
@Override
public void setWidth(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override
public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}
```

- Run kembali test Step 14.
- Tambahkan test s.setLength(3.0), lalu cek width dan length.
- Uji getArea() dan getPerimeter(). Apakah perlu dioverride di Square? Jangan menebak; buktikan dari output.

Checkpoint 12

Jelaskan mengapa setWidth()/setLength() perlu dioverride, tetapi getArea()/getPerimeter() dapat tetap memakai implementasi Rectangle.

Step 16 - Test hierarchy secara utuh (5 menit)

```
public class TestShape {
    public static void main(String[] args) {
        Shape a = new Shape("black", false);
        Circle b = new Circle(2.0, "blue", true);
        Rectangle c = new Rectangle(3.0, 4.0, "yellow", true);
        Square d = new Square(5.0, "green", true);

        System.out.println(a);
        System.out.println(b + " area=" + b.getArea());
        System.out.println(c + " area=" + c.getArea());
        System.out.println(d + " area=" + d.getArea());
    }
}
```

```
}  
}
```

Simpan output akhir. Buat diagram sederhana hierarchy class yang sesuai dengan code Anda.

5. Kembangkan Kasus Tugas Sebelumnya

Sekarang terapkan konsep yang baru Anda alami pada source code kasus pilihan dari tugas sebelumnya. Jangan mengganti kasus hanya agar inheritance mudah dibuat. Inheritance harus lahir dari kemiripan dan perbedaan yang memang ada pada model Anda.

5.1 Step 17 - Cari kandidat generalization (5 menit)

- Daftar minimal dua class yang memiliki state/behavior mirip.
- Tuliskan state/behavior yang sama.
- Tuliskan perbedaan masing-masing class.
- Buat kalimat: "Subclass adalah sebuah Superclass". Jika kalimat terasa tidak masuk akal, jangan gunakan inheritance.

5.2 Step 18 - Gambar sebelum coding (5 menit)

Gambar hierarchy sederhana. Tandai: superclass, subclass, state/behavior umum, state/behavior spesifik, dan method yang kemungkinan dioverride.

5.3 Step 19 - Implementasikan perubahan (15 menit)

- Minimal 1 superclass dan 2 subclass yang masuk akal.
- Subclass menggunakan extends.
- Constructor subclass memanggil super(...).
- Minimal 1 method dioverride menggunakan @Override.
- Minimal 1 override memanfaatkan super.method(...) secara bermakna, atau tulis alasan teknis jika memang tidak dibutuhkan.
- Encapsulation tetap dijaga; field private tidak diubah menjadi public.
- Program utama membuat object dari minimal dua subclass dan menunjukkan output/behavior yang berbeda.
- Fungsi utama tugas sebelumnya tetap berjalan setelah refactor.

5.4 Step 20 - Uji sebelum menyatakan selesai (10 menit)

ID	Skenario	Expected Result
T4-01	Object subclass dibuat melalui constructor	Bagian superclass dan subclass terinisialisasi benar
T4-02	Method warisan dipanggil dari subclass	Behavior superclass tersedia tanpa copy code
T4-03	Method override dipanggil	Behavior subclass yang dijalankan
T4-04	Override memakai super.method(...)	Bagian behavior superclass tetap digunakan
T4-05	Akses state mengikuti encapsulation	Tidak mengakses field private secara langsung
T4-06	Dua subclass diuji	Perbedaan behavior terlihat pada output
T4-07	Regression tugas lama	Fungsi utama sebelumnya tetap berjalan

6. Laporan dan Pengumpulan

6.1 Isi Laporan PDF

- Identitas: nama, NIM, kelas.
- Bagian A - Circle dan Cylinder: code inti, output TestCylinder, bukti constructor chain, bukti akses private ditolak, override getArea(), bug getVolume(), perbaikan super.getArea(), dan override toString().
- Bagian B - Shape hierarchy: diagram Shape-Circle/Rectangle-Square, code inti, output, dan bukti Square sebelum/sesudah override setter.
- Bagian C - Kasus tugas sebelumnya: diagram sebelum/sesudah, alasan memilih inheritance, code extends, super(...), @Override, super.method(...), dan output pengujian.
- Tabel skenario uji T4-01 s.d. T4-07 dengan status PASS/FAIL dan bukti.
- Jawaban refleksi pada Bagian 7.3.

6.2 Format Bukti

- Potongan code harus terbaca; jangan hanya screenshot IDE satu layar penuh.
- Output/error yang penting diberi caption singkat: apa yang diuji dan apa kesimpulannya.
- Jika ada error eksperimen, simpan bukti sebelum diperbaiki.
- Pastikan source final sama dengan source yang dijelaskan pada PDF.

6.3 Struktur Folder

```
NIM_Nama_PBO4
├── Laporan_PB04.pdf
├── source/
│   ├── Circle.java
│   ├── Cylinder.java
│   ├── TestCylinder.java
│   ├── Shape.java
│   ├── Rectangle.java
│   ├── Square.java
│   ├── TestShape.java
│   └── kasus-sebelumnya/
```

7. Checkpoint dan Refleksi

7.1 Definition of Done

1. Semua Step 1-16 telah dicoba, bukan hanya code final disalin.
2. Output dan minimal satu error eksperimen tersimpan sebagai bukti.
3. Saya dapat menunjukkan method yang diwarisi dan method yang dioverride.
4. Saya dapat menjelaskan urutan constructor dan fungsi super(...).
5. Saya dapat menjelaskan mengapa super.getArea() diperlukan pada getVolume().
6. Saya dapat menjelaskan mengapa Square perlu override setter.
7. Kasus tugas sebelumnya telah direfactor dan lulus skenario uji utama.
8. PDF dan source siap di github.

7.2 Self Assessment

Pernyataan	Skor 1-5
Saya dapat menjelaskan extends dari code yang saya jalankan.	

Pernyataan	Skor 1-5
Saya dapat membedakan inherited method dan overridden method.	
Saya memahami mengapa constructor subclass memanggil super(...).	
Saya dapat menggunakan super.method(...) dengan alasan yang tepat.	
Saya dapat menjaga encapsulation pada hierarchy class.	
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	

7.3 Refleksi - Jawab dari pengalaman praktikum

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?
2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?
3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?
4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?
5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?
6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?
7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.
8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."